



Data

Model

Evaluate

Redesign

Evaluation, Experimentation & AI Software Design

AI-Based Programming

Measuring · Debugging · Iterating · Architecture

Learning Goals



Define Evaluation

Understand evaluation as a full pipeline, not a single metric



Software Patterns

Apply design patterns to AI components for clean architecture



Design Experiments

Create reproducible experiments with control variables



Connect & Improve

Link evaluation results back to system improvement strategies



Error Analysis

Perform error analysis to identify and understand failure modes

Why Evaluation Matters

✓ Good Test Score

- High accuracy on test set
- Passes benchmark metrics
- Looks great on paper
- Clean confusion matrix



✗ Bad Real-World Behaviour

- Fails on edge cases
- Biased toward majority class
- Unsafe on rare inputs
- No monitoring in production

A model must be judged by correctness, robustness, and safe behaviour on edge cases.

Evaluation Pipeline



↩ Results inform next experiment – evaluation is iterative

Input Stage

Define the task clearly.
Version and split the dataset reproducibly.

Core Loop

Train baseline, compute metrics (F1, accuracy), and inspect the confusion matrix.

Action Stage

Compare against prior runs, then decide the next change to model or data.

Metrics & Formulas

Accuracy

$$(TP+TN)/(Total)$$

Misleading on imbalanced data

Precision

$$TP / (TP+FP)$$

Of all predicted positives, how many are correct?

Recall

$$TP / (TP+FN)$$

Of all actual positives, how many are found?

F1 Score

$$2 \cdot P \cdot R / (P+R)$$

Harmonic mean of precision and recall

Confusion Matrix

		Predicted	
		Pos	Neg
Actual	Pos	TP True Positive	FN False Negative
	Neg	FP False Positive	TN True Negative

$$F1 = 2TP / (2TP + FP + FN)$$

Confusion Matrix – Code Example



```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
```

```
y_true = [1, 0, 1, 1, 0, 1]
y_pred = [1, 0, 0, 1, 0, 1]
```

```
cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap="Blues")
plt.show()
```

Why Use It?

Better than a single score — reveals the shape of errors, not just their count.

TP + TN

Correct predictions. These are what we want to maximise.

FP + FN

Prediction errors. Identify which type matters more for your task.

sklearn.metrics.confusion_matrix() is the quickest path to actionable error insight.

When Accuracy Is Misleading

95%

Accuracy

But the model is
useless!

Class Distribution

Negative (95%)

Positive (5%)

The Problem

If only 5% of samples are positive, always predicting 'negative' gives 95% accuracy — but zero useful predictions.

Better Metrics

Use Precision, Recall, and F1 for honest performance. They expose the model's real behaviour on each class.

Key Takeaway

Never rely on a single metric. The evaluation strategy must match the class distribution and task requirements.

Experiment Design

A – Baseline (Control)

- Fixed data split
- Original hyperparameters
- Standard preprocessing
- Logged with config v1

VS

B – Experiment (Treatment)

- SAME data split
- **ONE changed parameter**
- Same preprocessing
- Logged with config v2

⚠ Changing multiple things at once makes results impossible to interpret.

Reproducibility Checklist



Version Code with Git

Every experiment should point to a commit. Tag releases for paper/demo checkpoints.



Version Datasets & Artifacts

Use DVC or commit data hashes. The same model + different data = different result.



Fix Random Seeds

Set seeds in NumPy, PyTorch, and Python random. Ensures same shuffle & init.



Pin Package Versions

requirements.txt or environment.yml with exact versions. Avoids silent breakage.



Log Parameters & Metrics

Use MLflow, or a simple YAML log. You will forget what you tried last week.



Save Experiment Config

Store the full config alongside the checkpoint so reruns are one command away.

Configuration-Driven Experiments

```
import yaml
import mlflow

with open("config.yaml") as f:
    cfg = yaml.safe_load(f)

mlflow.set_experiment(cfg["experiment_name"])

with mlflow.start_run():
    mlflow.log_params(cfg["model"])
    mlflow.log_param("train_size",
                    cfg["data"]["train_size"])
    mlflow.log_metric("f1", 0.87)
```

config.yaml

```
experiment_name: run_01

model:
  lr: 0.001
  epochs: 30

data:
  train_size: 0.8
```

Benefits

- Rerun any experiment with one command
- Audit differences between runs easily
- No accidental hardcoded changes
- MLflow tracks params, metrics & artifacts

Error Analysis

Where does the model fail – and why?



Noisy Labels

Ground truth errors in the dataset confuse the model and hide true performance.



Rare Classes

The model sees few examples of minority classes and can't learn them well.



Edge Cases

Unusual inputs outside the training distribution cause unexpected failures.



Distribution Shift

Production data differs from training data. Model degrades silently over time.

Inspect individual examples manually — aggregate metrics hide patterns.

Error Analysis — Example Table

#	True Label	Predicted	Correct?	Issue / Note
1	Positive	Negative	✗	False negative on rare class — model undertrained on minority
2	Negative	Positive	✗	Boundary confusion — sample near decision boundary
3	Positive	Positive	✓	Correct prediction — clear positive signal
4	Negative	Negative	✓	Correct prediction — clear negative signal
5	Positive	Negative	✗	Distribution shift — test sample unlike training examples
6	Negative	Positive	✗	Noisy label — possible annotation error in ground truth

Recurring patterns — false negatives on rare class — become actionable improvement ideas.

Design Patterns for AI Components

Design patterns separate concerns and make AI systems easier to maintain and extend.



Factory

When to use

Instantiating different models from a config file.

```
create_model("random_forest")
```

Extensible pipelines — add new algorithms without changing calling code.



Strategy

When to use

Swapping algorithms, decoders, or evaluation routines at runtime.

```
strategy = F1Strategy()  
strategy.score(y_true, y_pred)
```

Interchangeable logic — same interface, different implementation.



Observer

When to use

Multiple components react to events: end-of-epoch, drift detected.

```
trainer.on("epoch_end",  
           [logger, checkpoint, dashboard])
```

Decoupled monitoring — no tangled code in training loops.

Factory Pattern in AI Systems

```
config.yaml  
experiment: run_03
```

Factory

Reads config → creates objects

```
model = XGBoost()
```

```
class ModelFactory:  
    @staticmethod  
    def create(model_type):  
        if model_type == "xgb":  
            return XGBoostModel()  
        elif model_type == "rf":  
            return RandomForestModel()
```

```
create_model("random_forest")
```

```
create_metric("f1")
```

```
create_dataset("csv")
```

Centralised Creation

All object creation in one place — no hardcoded class names scattered across the codebase.

Extensible

Add a new model type by registering it — no changes to callers or pipeline code needed.

Config-Driven

The config file drives which classes are instantiated. Swap models without touching code.

Strategy Pattern – Code Example

```
if loss == "mse":  
    ...  
elif loss == "mae":  
    ...
```



```
class LossStrategy:  
    def compute(self, y_true, y_pred):  
        pass  
  
class MSE(LossStrategy):  
    def compute(self, y_true, y_pred):  
        return ((y_true - y_pred) ** 2).mean()  
  
class MAE(LossStrategy):  
    def compute(self, y_true, y_pred):  
        return abs(y_true - y_pred).mean()
```

EvalStrategy
(Abstract)



MSE Strategy

MAE Strategy

Evaluator
(uses strategy)

Observer Pattern in AI Pipelines



Mini Case Study — Classification Pipeline

BEFORE

Accuracy: **92%** *Looks impressive*

Recall: **34%** *Very weak!*

F1 Score: **0.41** *Much worse than accuracy implies*

Root cause: model misses minority-class samples

→
Redesign

AFTER

Accuracy: **91%** *Slightly lower — expected*

Recall: **78%** *Huge improvement!*

F1 Score: **0.83** *Much more reliable*

Fix: class balancing + threshold tuning + better eval pipeline

Practical Assignment



1

Choose a Dataset

Pick a simple classification dataset (Iris, Titanic, or your own).



2

Train Baseline

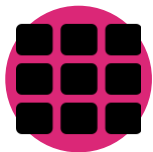
Fit a straightforward model (decision tree, logistic regression).



3

Evaluate with 2+ Metrics

Compute at least accuracy and F1. Use sklearn.metrics.



4

Confusion Matrix

Plot and interpret the matrix. Identify FP and FN patterns.



5

Error Analysis

Write 3 observations about where the model fails and why.



6

Software Improvement

Propose one design improvement: pattern, config, or pipeline change.

Evaluation · Experimentation & Software Design Work Together

Measurable

Good AI systems are always evaluated with the right metrics for the task.

Reproducible

Every run is versioned, logged, and repeatable. No mystery experiments.

Debuggable

Error analysis turns confusion matrices into concrete improvement actions.

Well-Architected

Patterns like Factory, Strategy & Observer keep pipelines clean and flexible.